

Lab 2 - Polling and Interrupts

Module	ET4047
Date	20/09/2026
Lab No.	Lab 2
Author	Luke Griffin

Introduction

In this lab we explore two fundamental approaches for handling input events in embedded systems: polling and interrupts. We use an Arduino (Uno or Nano) with a breadboard as our platform. By the end of the lab you will understand what polling and interrupts are, and you will work through a series of short exercises that build on each other using push buttons and LEDs.

Safety and Handling Precautions

Read this section before you touch any equipment. Following these steps keeps you safe and protects the boards and components at your bench.

Important - please read carefully:

- **You pay for what you break.** The boards and components belong to the department. If you damage a part through careless or incorrect wiring, you may have to replace or pay for it - so work slowly and carefully.
- **Power off before wiring.** Always unplug the Arduino from the USB / power lead before you add, remove, or change any connection. Only plug it back in once the circuit is finished and checked.
- **Keep off metal surfaces.** Do not build or rest your circuit on a metal or other conductive surface - bare wires and component legs can short together and damage the board.
- **Double-check the wiring.** Before powering on, check every connection against the wiring diagram, especially 5V and GND. Never connect 5V straight to GND, and never wire an LED without its resistor.
- **Mind polarity and resistors.** LEDs only work one way round (the long leg is positive). Use the stated values - 220 ohm in series with each LED.
- **Handle boards with care.** Hold boards by the edges and avoid touching the chip pins; static electricity can damage the electronics.
- **Watch for trouble.** If a part gets hot, smells of burning, or you see smoke, unplug immediately and tell a demonstrator. Do not keep using it.
- **Do not force parts.** Push wires and components gently into the breadboard - forcing them bends legs and breaks components.
- **Keep the bench clear.** No drinks or liquids near the equipment, and keep your work area tidy.
- **Ask if unsure.** If you are not certain your circuit is correct, ask a demonstrator to check it before you power on.

What is Polling?

Polling is a method where the CPU continuously checks the status of an input in a loop to see if an event has occurred. The microcontroller repeatedly reads a button's state in the main loop and acts only when it detects a change, such as a button press. Polling is simple to implement using if statements inside loop(), but it has trade-offs:

- Simplicity - easy to write and reason about.
- Constant CPU usage - the processor is always busy checking.
- Latency - a response can be delayed until the next loop iteration.
- Predictability - behaviour is deterministic and easy to follow.

In summary, polling is like a teacher repeatedly asking every student whether they have a question - systematic, but potentially inefficient.

What are Interrupts?

Interrupts allow an external event to signal the CPU when attention is needed, pausing the current program to handle that event immediately. Instead of the CPU constantly checking the button, the button hardware interrupts the CPU when pressed. The CPU runs a short Interrupt Service Routine (ISR) to handle the press, then returns to the main program. Key points:

- Efficient and responsive - no time wasted checking idle inputs.
- Low latency - the event is handled almost immediately.
- Complexity - requires more careful design than polling.
- Priority and interference - ISRs must be kept short.

In summary, using interrupts is like students raising their hands when they have a question - the teacher (CPU) does not have to check every student and can respond immediately.

Lab Objective

We build up in small steps, one exercise at a time:

- **Exercise 1 - Getting started:** the Arduino essentials - setup(), loop(), pin modes, digitalWrite() and the Serial Monitor.
- **Exercise 2 - Debounce:** read a button cleanly by removing contact bounce in software.
- **Exercise 3 - Polling:** continuously check a button in the main loop.
- **Exercise 4 - Interrupts:** react to a button with a hardware interrupt and an ISR.
- **Exercise 5 - Two buttons:** a second button controls a second LED.

Hardware Setup

Most exercises use the same breadboard circuit - physical push buttons and physical LEDs wired to the Arduino (no shield). Exercise 1 uses only the two LEDs. Each exercise below has its own wiring diagram; the standard parts are:

- **2 LEDs:** connect LED 1's long leg (anode) to D4 and LED 2's long leg to D5. Connect each LED's short leg (cathode) to GND through its own 220 ohm resistor.
- **Push button S1:** one leg to D2, the other leg to GND. Used from Exercise 2 on.
- **Push button S2:** one leg to D3, the other leg to GND. Used in Exercise 5 only.

All buttons use INPUT_PULLUP, so a pin reads HIGH when the button is idle and LOW when pressed (active-low) - no external resistor is needed on the button. Pins D2 and D3 are used for the buttons because these are the Arduino's external interrupt pins, which the interrupts exercise (Exercise 4) relies on.

Polling vs. Interrupts in Microcontrollers

Polling

CPU repeatedly to checks the status rf a device

```
graph TD; Start([Start]) --> Check{Check device}; Check -- No --> Check; Check -- Yes --> Action[Perform action];
```

Feature	Polling	Interrupts
CPU Usage	Always active	Only when event occurs
Complexity	Simple	More complex
Power Efficiency	Low	High
Response Time	Depends on loop speed	Immediate (hardware-driven) Excellent
Scalability	Poor	

When to Use Polling

- Simple or infrequent tasks

When to Use Interrupts

- Real-time or asynchronous

```
graph TD; Main([Main rupt]) -- Interrupt --> Service[Interrupt service routine];
```

Exercise 1: Getting Started

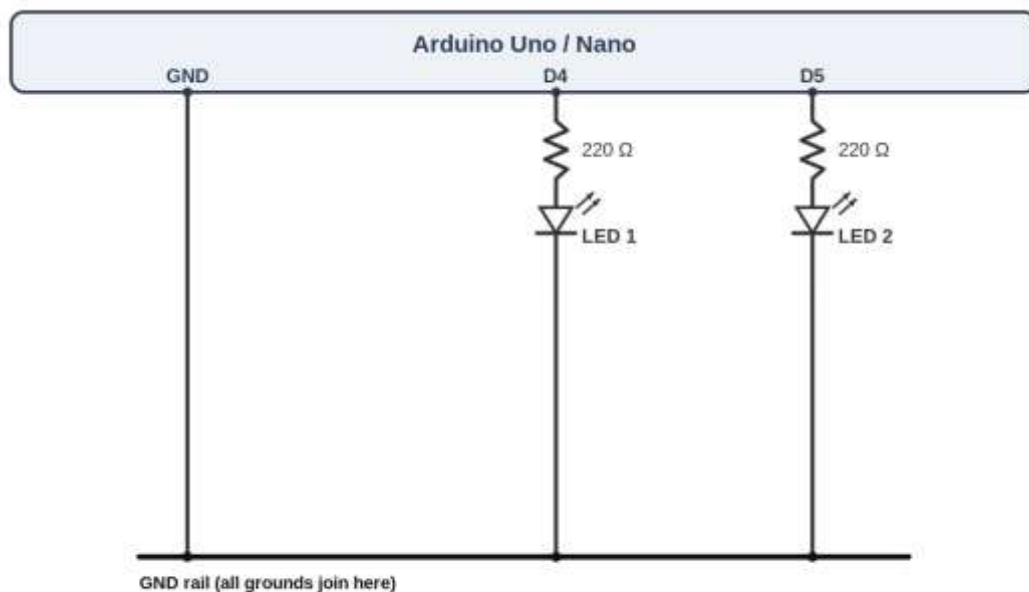
Before using any buttons, we get familiar with the Arduino essentials: the `setup()` and `loop()` functions, setting pin modes, driving outputs with `digitalWrite()`, and printing to the Serial Monitor. This program blinks two LEDs alternately and prints what it is doing.

Key ideas

- `setup()` runs once when the board powers on or resets. It is where you initialise things - open the serial port and set pin modes.
- `loop()` runs straight after `setup()` and then repeats forever. This is where the ongoing work happens.
- `pinMode(pin, OUTPUT)` tells the Arduino a pin will drive something such as an LED; use `INPUT` (or `INPUT_PULLUP`) to read a pin.
- `digitalWrite(pin, HIGH/LOW)` sets an output pin to 5V (HIGH) or 0V (LOW) - this is what turns an LED on or off.
- `Serial.begin(9600)` opens the Serial Monitor at 9600 baud, and `Serial.println(...)` prints a line you can read on your PC (open Tools > Serial Monitor and set it to 9600 baud).

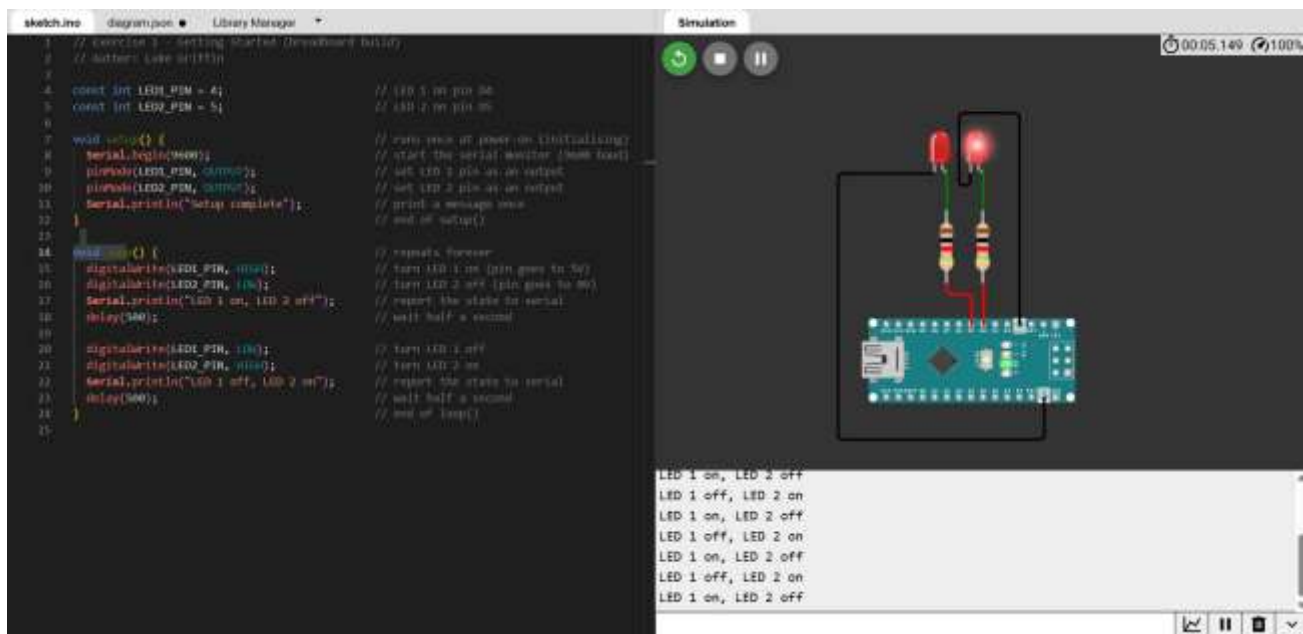
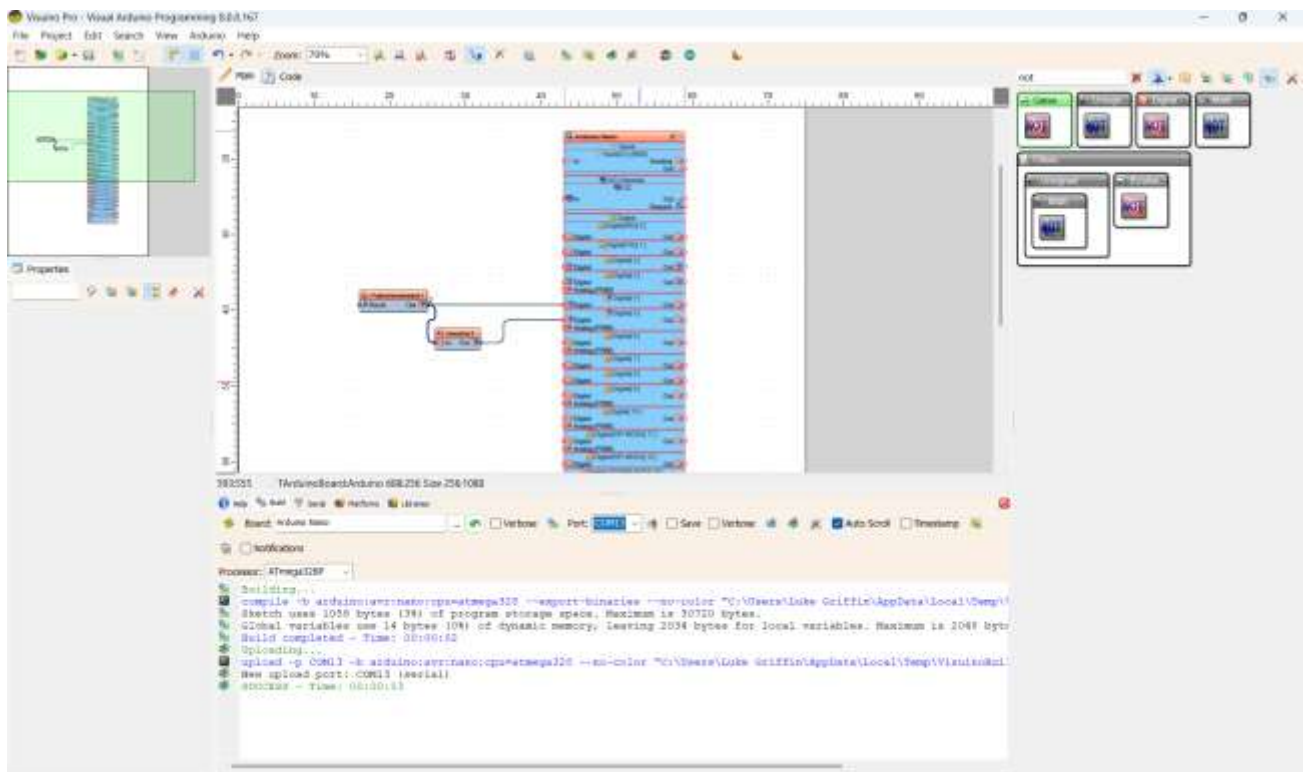
Circuit and Setup

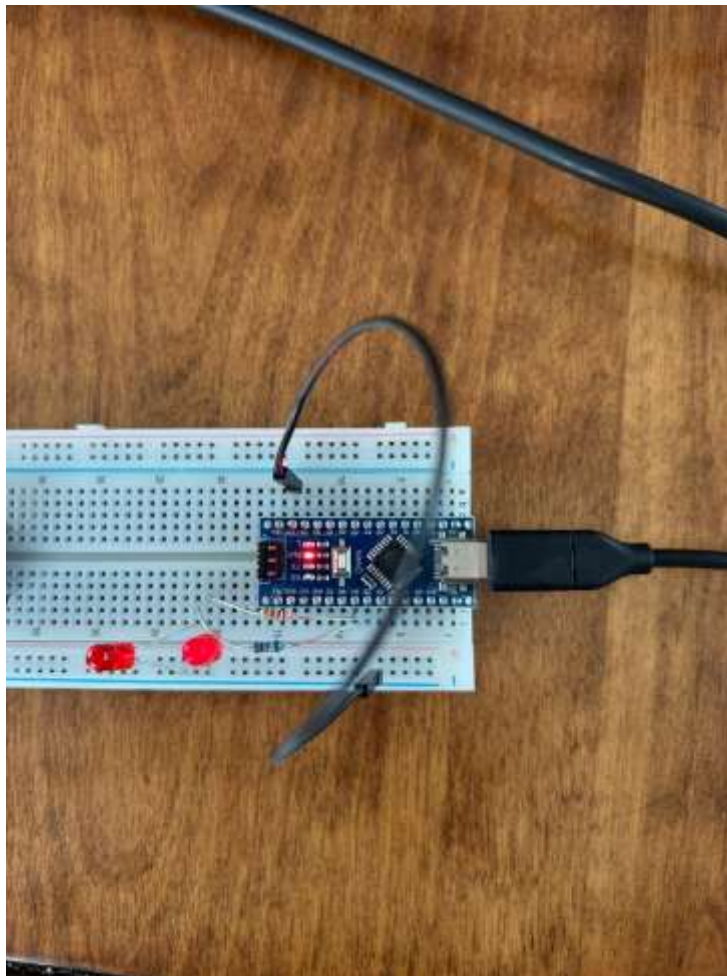
Wiring diagram - build this circuit on the breadboard.



Not to scale. Buttons use INPUT_PULLUP (idle HIGH, pressed connects the pin to GND = LOW). LED long leg goes to the pin side.

Figure 1a: Reference wiring for Exercise 1.





Code

```
// Exercise 1 - Getting Started (breadboard build)
// Author: Luke Griffin

const int LED1_PIN = 4;           // LED 1 on pin D4
const int LED2_PIN = 5;           // LED 2 on pin D5

void setup() {                    // runs once at power-on (initialising)
  Serial.begin(9600);             // start the serial monitor (9600 baud)
  pinMode(LED1_PIN, OUTPUT);      // set LED 1 pin as an output
  pinMode(LED2_PIN, OUTPUT);      // set LED 2 pin as an output
  Serial.println("Setup complete"); // print a message once
}                                  // end of setup()

void loop() {                     // repeats forever
  digitalWrite(LED1_PIN, HIGH);    // turn LED 1 on (pin goes to 5V)
  digitalWrite(LED2_PIN, LOW);     // turn LED 2 off (pin goes to 0V)
  Serial.println("LED 1 on, LED 2 off"); // report the state to serial
  delay(500);                     // wait half a second

  digitalWrite(LED1_PIN, LOW);     // turn LED 1 off
  digitalWrite(LED2_PIN, HIGH);    // turn LED 2 on
  Serial.println("LED 1 off, LED 2 on"); // report the state to serial
  delay(500);                     // wait half a second
}                                  // end of loop()
```

How it works

- `setup()` opens the serial monitor and sets both LED pins as outputs - this is the initialising step.
- `loop()` turns LED 1 on and LED 2 off, prints the state, waits half a second, then swaps them, so the two LEDs alternate forever.
- Open the Serial Monitor at 9600 baud to watch the messages printed each time the state changes.

Wokwi: you can also build this in the simulator at wokwi.com. Start a new Arduino Uno project, drag in a breadboard and draw the circuit shown in the wiring diagram (a breadboard and 2 LEDs with resistors on D4 and D5), paste the sketch above, and press Play.

Exercise 2: Button Debounce

This exercise introduces debounce: making a single button press register only once. It uses one push button (S1) and one LED. Each clean press toggles the LED on or off.

Button wiring (active-low)

Like every button in this lab, S1 uses INPUT_PULLUP. The internal pull-up resistor holds the pin HIGH when the button is idle, and pressing the button connects the pin to GND, pulling it LOW. This is called active-low: pressed reads LOW. No external resistor is needed on the button.

What is debounce?

A mechanical push button does not switch cleanly. For a few milliseconds after it is pressed or released the metal contacts bounce, making the pin flip between HIGH and LOW several times. Without handling this, one physical press can be read as many presses. Debounce fixes it: we only accept a new button state once the reading has stayed the same for a short settling time (here 50 ms).

Circuit and Setup

One push button (S1 on D2) and one LED (LED 1 on D4).

Wiring diagram - build this circuit on the breadboard.

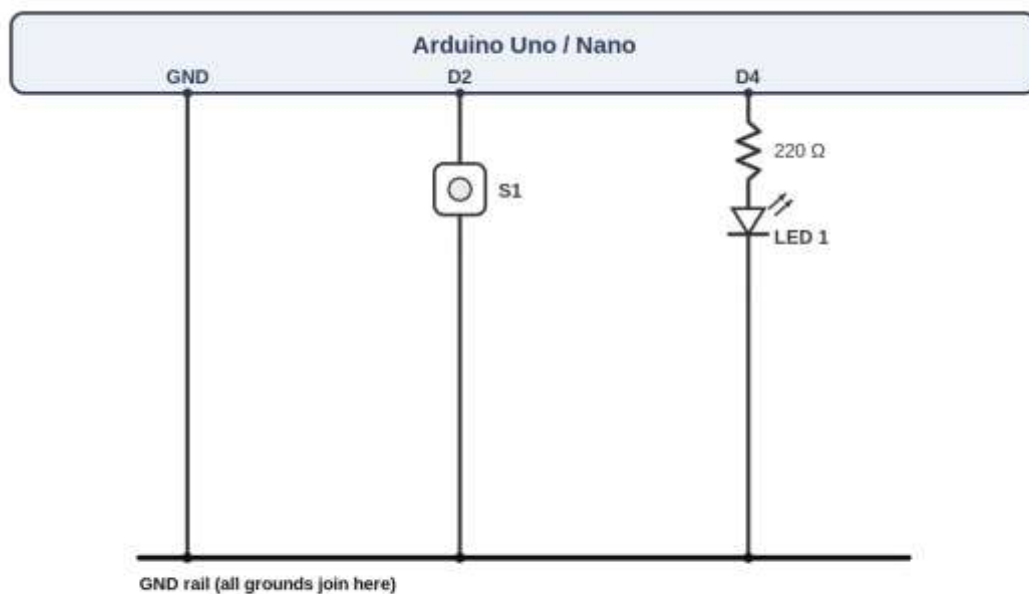


Figure 2a: Reference wiring for Exercise 2.

Wokwi screenshot - paste an image of your Wokwi circuit in the box below.

[Wokwi screenshot - Exercise 2 circuit - paste here]

Physical setup - paste a photo of your breadboard wiring in the box below.

[Photo - Exercise 2 breadboard setup - paste here]

Code

```
// Exercise 2 - Button Debounce (breadboard build)
// Author: Luke Griffin

const int BUTTON1_PIN = 2;           // S1 push button on pin D2
const int LED1_PIN = 4;              // LED 1 on pin D4

int buttonState = HIGH;              // current debounced state (idle HIGH)
int lastReading = HIGH;              // previous raw reading (idle HIGH)
unsigned long lastChangeTime = 0;    // time the raw reading last changed
const unsigned long debounceDelay = 50; // wait 50 ms for it to settle
bool ledOn = false;                 // remembers if LED 1 is on

void setup() {                       // runs once at power-on
  Serial.begin(9600);                // open serial monitor (9600 baud)
  pinMode(BUTTON1_PIN, INPUT_PULLUP); // pull-up; idle HIGH, pressed LOW
  pinMode(LED1_PIN, OUTPUT);          // LED 1 pin is an output
  digitalWrite(LED1_PIN, LOW);        // start with LED 1 off
}

void loop() {                        // repeats forever
  int reading = digitalRead(BUTTON1_PIN); // read S1 (LOW = pressed)

  if (reading != lastReading) {      // raw reading changed (a bounce?)
    lastChangeTime = millis();       // note the time of the change
  }

  if ((millis() - lastChangeTime) > debounceDelay) { // steady long enough?
    if (reading != buttonState) {    // stable state actually changed
      buttonState = reading;         // save the new stable state
      if (buttonState == LOW) {      // a clean press (active-low)
        ledOn = !ledOn;             // flip the LED memory
        digitalWrite(LED1_PIN, ledOn); // apply it to LED 1
        Serial.println("Debounced press -> LED toggled"); // print status
      }
    }
  }

  lastReading = reading;             // remember reading for next pass
}
```

How it works

- Each pass the raw button reading is taken. Whenever it changes, a timer (lastChangeTime) is reset.
- Only when the reading has been steady for longer than debounceDelay (50 ms) is it accepted as the real state.
- On a clean change to LOW (a genuine press, active-low) LED 1 is toggled once and a message is printed.
- Try setting debounceDelay to 0 - you will often see the LED toggle several times for a single press, which is the bounce that debounce removes.

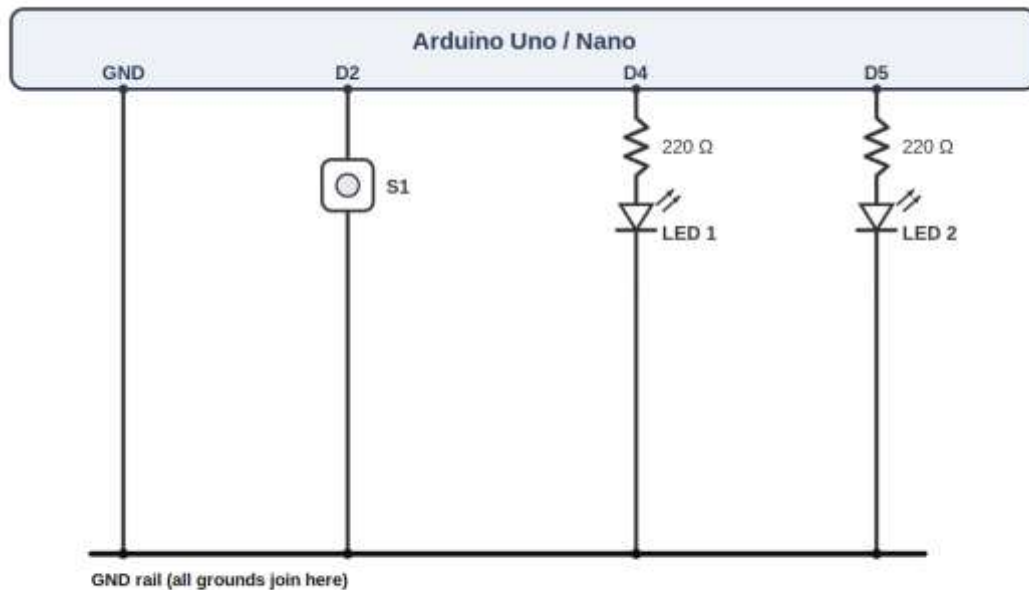
Wokwi: you can also build this in the simulator at wokwi.com. Start a new Arduino Uno project, drag in a breadboard and draw the circuit shown in the wiring diagram (S1 on D2 to GND, and LED 1 with a 220 ohm resistor on D4), paste the sketch above, and press Play.

Exercise 3: Polling

Use polling to detect when the button is pressed, then turn both LEDs on for a short duration. The Arduino continuously loops, checking the state of the button. No interrupts are used.

Circuit and Setup

Wiring diagram - build this circuit on the breadboard.



Not to scale. Buttons use INPUT_PULLUP (idle HIGH, pressed connects the pin to GND = LOW). LED long leg goes to the pin side.

Figure 3a: Reference wiring for Exercise 3.

Wokwi screenshot - paste an image of your Wokwi circuit in the box below.

[Wokwi screenshot - Exercise 3 circuit - paste here]

Physical setup - paste a photo of your breadboard wiring in the box below.

[Photo - Exercise 3 breadboard setup - paste here]

Code

```
// Exercise 3 - Polling (breadboard build)
// Author: Luke Griffin

const int BUTTON1_PIN = 2;           // S1 push button on pin D2
const int LED1_PIN = 4;              // LED 1 on pin D4
const int LED2_PIN = 5;              // LED 2 on pin D5

void setup() {                       // runs once at power-on
  Serial.begin(9600);                // open serial monitor (9600 baud)
  pinMode(BUTTON1_PIN, INPUT_PULLUP); // idle HIGH, pressed LOW
  pinMode(LED1_PIN, OUTPUT);          // LED 1 pin is an output
  pinMode(LED2_PIN, OUTPUT);          // LED 2 pin is an output
  digitalWrite(LED1_PIN, LOW);         // start with LED 1 off
  digitalWrite(LED2_PIN, LOW);         // start with LED 2 off
}

void loop() {                         // repeats forever
  bool s1 = (digitalRead(BUTTON1_PIN) == LOW); // true when S1 is pressed

  if (s1) {                           // if the button is pressed
    digitalWrite(LED1_PIN, HIGH);      // turn LED 1 on
    digitalWrite(LED2_PIN, HIGH);      // turn LED 2 on
    Serial.println("Button S1 pressed -> LEDs ON"); // print status
    delay(3000);                       // keep LEDs on for 3 s
  } else {                             // otherwise (not pressed)
    digitalWrite(LED1_PIN, LOW);        // turn LED 1 off
    digitalWrite(LED2_PIN, LOW);        // turn LED 2 off
    Serial.println("Button S1 released -> LEDs OFF"); // print status
  }

  delay(300);                          // short pause; keeps serial readable
}
```

Expected Output



```
Task1_lab2.ino
16
17 void loop() {
18   // poll button (active low)
19   bool s1 = (digitalRead(BUTTON1_PIN) == LOW);
20
21   if (s1) {
22     for (int i = 0; i < 10; i++) digitalWrite(ledPins[i], HIGH);
23     Serial.println("Button S1 pressed -> LEDs ON");
24   } else {
25     for (int i = 0; i < 10; i++) digitalWrite(ledPins[i], LOW);
26     Serial.println("Button S1 released -> LEDs OFF");
27   }
28
29   delay(500); // slow down prints so serial monitor stays readable
30 }
31
```

Serial Monitor X

Message (click to send message to 'Arduino Nano Every' on COM7)

New Line 9600 baud

Button S1 released -> LEDs OFF
Button S1 released -> LEDs OFF
Button S1 pressed -> LEDs ON
Button S1 pressed -> LEDs ON
Button S1 pressed -> LEDs ON
Button S1 pressed -> LEDs ON
Button S1 released -> LEDs OFF

Ln 22, Col 18 Arduino Nano Every on COM7

Figure 3b: Serial monitor output for Exercise 3.

How it works

- `loop()` reads the button each pass. If it is pressed, both LEDs are set HIGH and held for 3 seconds; otherwise they are set LOW.
- The `delay(300)` at the end slows the loop so the serial monitor stays readable.
- This is polling in action - the program flow is dominated by actively watching the input.

Wokwi: you can also build this in the simulator at wokwi.com. Start a new Arduino Uno project, drag in a breadboard and draw the circuit shown in the wiring diagram, paste the sketch above, and press Play.

Exercise 4: Interrupts

Re-implement the same behaviour using an external interrupt. The Arduino no longer checks the button in the main loop. Instead, pressing the button triggers an ISR that sets a flag, and the main loop responds to that flag.

Circuit and Setup

Same wiring as Exercise 3 - one push button (S1) and two LEDs.

Wiring diagram - build this circuit on the breadboard.

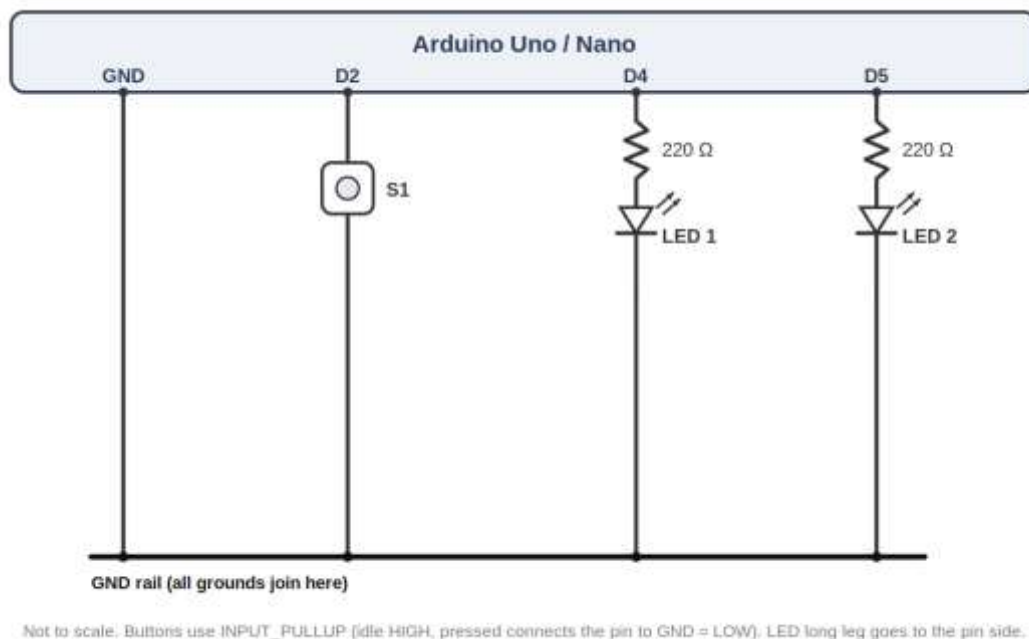


Figure 4a: Reference wiring for Exercise 4.

Wokwi screenshot - paste an image of your Wokwi circuit in the box below.

[Wokwi screenshot - Exercise 4 circuit - paste here]

Physical setup - paste a photo of your breadboard wiring in the box below.

[Photo - Exercise 4 breadboard setup - paste here]

Code

```
// Exercise 4 - Interrupts (breadboard build)
// Author: Luke Griffin

const int BUTTON1_PIN = 2;           // S1 on pin D2 (interrupt pin)
const int LED1_PIN = 4;              // LED 1 on pin D4
const int LED2_PIN = 5;              // LED 2 on pin D5

volatile bool s1Flag = false;        // flag set by the ISR

void setup() {                       // runs once at power-on
  Serial.begin(9600);                // open serial monitor (9600 baud)
  Serial.println("Program started - waiting for S1..."); // startup message
  pinMode(BUTTON1_PIN, INPUT_PULLUP); // idle HIGH, pressed LOW
  pinMode(LED1_PIN, OUTPUT);          // LED 1 pin is an output
  pinMode(LED2_PIN, OUTPUT);          // LED 2 pin is an output
  digitalWrite(LED1_PIN, LOW);         // start with LED 1 off
  digitalWrite(LED2_PIN, LOW);         // start with LED 2 off
  // link the button to the ISR (fires on the HIGH->LOW press)
  attachInterrupt(digitalPinToInterrupt(BUTTON1_PIN), handleS1ISR, FALLING); // attach ISR
}

void loop() {                        // repeats forever
  if (s1Flag) {                     // did the ISR fire?
    s1Flag = false;                 // clear the flag
    Serial.println("S1 interrupt detected - LEDs ON"); // print status
    digitalWrite(LED1_PIN, HIGH);   // turn LED 1 on
    digitalWrite(LED2_PIN, HIGH);   // turn LED 2 on
    delay(3000);                     // keep LEDs on for 3 s
    Serial.println("3 seconds elapsed - LEDs OFF"); // print status
    digitalWrite(LED1_PIN, LOW);     // turn LED 1 off
    digitalWrite(LED2_PIN, LOW);     // turn LED 2 off
  }
  // main loop is otherwise idle
}

void handleS1ISR() {                 // interrupt service routine
  s1Flag = true;                     // keep it short: set the flag
}
```

Expected Output

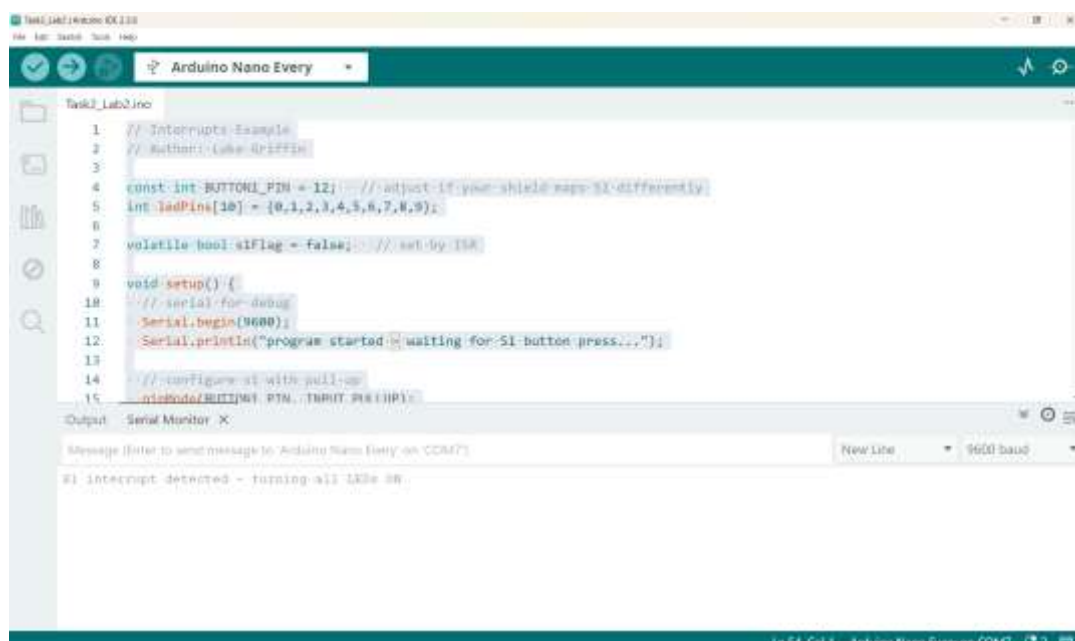


Figure 4b: Serial monitor output for Exercise 4.

How it works

- `volatile bool s1Flag` signals from the ISR to the main loop; volatile ensures it is always read correctly.
- `attachInterrupt(... FALLING)` runs the ISR on the falling edge (HIGH to LOW = a press).
- `handleS1ISR()` is kept short: it only sets the flag, with no prints or delays.

Wokwi: you can also build this in the simulator at wokwi.com. Start a new Arduino Uno project, drag in a breadboard and draw the circuit shown in the wiring diagram, paste the sketch above, and press Play.

Exercise 5: Two Buttons

Add a second push button (S2 on D3), one button per LED. Below is a polling version for two buttons: S1 lights LED 1 and S2 lights LED 2.

Circuit and Setup

Same as before plus a second push button (S2 on D3).

Wiring diagram - build this circuit on the breadboard.

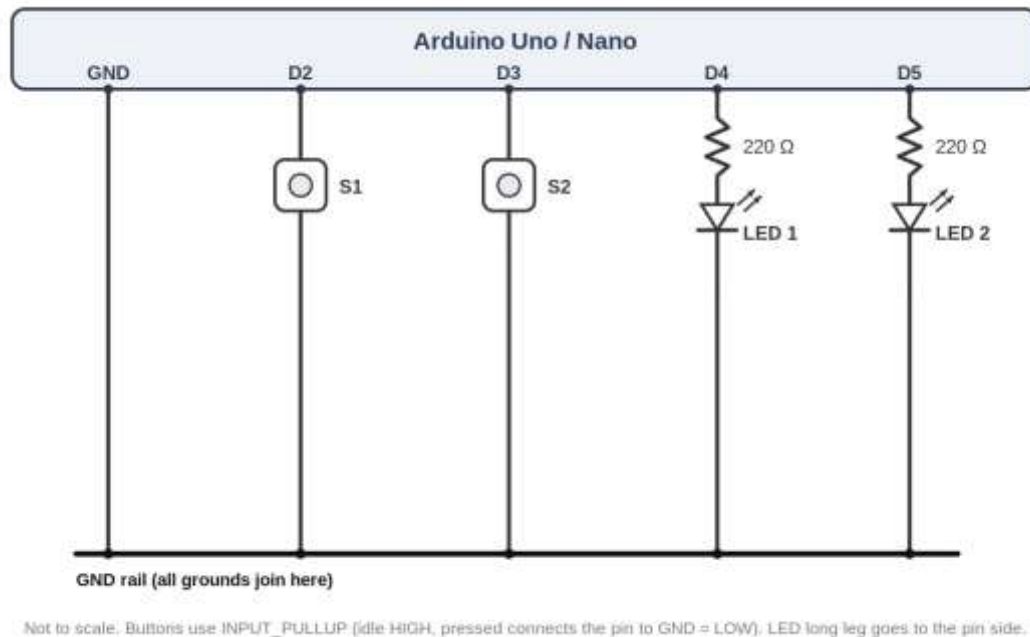


Figure 5a: Reference wiring for Exercise 5.

Wokwi screenshot - paste an image of your Wokwi circuit in the box below.

[Wokwi screenshot - Exercise 5 circuit - paste here]

Physical setup - paste a photo of your breadboard wiring in the box below.

[Photo - Exercise 5 breadboard setup - paste here]

Polling Code (provided)

```
// Exercise 5 - Polling, two buttons (breadboard build)
// Author: Luke Griffin

const int BUTTON1_PIN = 2;           // S1 push button on pin D2
const int BUTTON2_PIN = 3;           // S2 push button on pin D3
const int LED1_PIN = 4;              // LED 1 on pin D4
const int LED2_PIN = 5;              // LED 2 on pin D5

void setup() {                       // runs once at power-on
  Serial.begin(9600);                // open serial monitor (9600 baud)
  pinMode(BUTTON1_PIN, INPUT_PULLUP); // S1 idle HIGH, pressed LOW
  pinMode(BUTTON2_PIN, INPUT_PULLUP); // S2 idle HIGH, pressed LOW
  pinMode(LED1_PIN, OUTPUT);          // LED 1 pin is an output
  pinMode(LED2_PIN, OUTPUT);          // LED 2 pin is an output
  digitalWrite(LED1_PIN, LOW);         // start with LED 1 off
  digitalWrite(LED2_PIN, LOW);         // start with LED 2 off
}

void loop() {                         // repeats forever
  bool s1 = (digitalRead(BUTTON1_PIN) == LOW); // true when S1 is pressed
  bool s2 = (digitalRead(BUTTON2_PIN) == LOW); // true when S2 is pressed

  if (s1) {                           // if S1 is pressed
    digitalWrite(LED1_PIN, HIGH);      // turn LED 1 on
    Serial.println("S1 pressed -> LED 1 ON"); // print status
    delay(3000);                       // keep LED 1 on for 3 s
  } else {                             // otherwise
    digitalWrite(LED1_PIN, LOW);        // turn LED 1 off
  }

  if (s2) {                           // if S2 is pressed
    digitalWrite(LED2_PIN, HIGH);      // turn LED 2 on
    Serial.println("S2 pressed -> LED 2 ON"); // print status
    delay(3000);                       // keep LED 2 on for 3 s
  } else {                             // otherwise
    digitalWrite(LED2_PIN, LOW);        // turn LED 2 off
  }

  delay(50);                          // small pause between checks
}
```

Your Task

Using the polling code above and the interrupt code from Exercise 4, write an interrupt-driven version so that pressing Button 1 turns on LED 1 and pressing Button 2 turns on LED 2. Because S1 and S2 are on D2 and D3 (both interrupt pins), the same circuit works for both the polling and interrupt versions - only the code changes.

Wokwi: you can also build this in the simulator at wokwi.com. Start a new Arduino Uno project, drag in a breadboard and draw the circuit shown in the wiring diagram, paste the sketch above, and press Play.

Conclusion

In this lab we started from the Arduino basics and then compared polling and interrupts, using a simple buttons-and-LEDs breadboard circuit.

Summary of key learnings:

- The Arduino runs `setup()` once for initialising, then `loop()` forever; `pinMode`, `digitalWrite` and the Serial Monitor are the basic tools.
- Buttons use `INPUT_PULLUP` and are active-low: the pin idles HIGH and reads LOW when pressed, so no external resistor is needed.

- Debounce: mechanical buttons bounce, so a short settling delay (about 50 ms) makes each press register only once.
- Polling repeatedly asks the hardware "Are you pressed?" - simple, but can be inefficient.
- Interrupts let the hardware tell the software "Button pressed now!", so the CPU does not check unnecessarily.
- Interrupt-driven systems let the CPU do other work or sleep, which is ideal for power saving.
- Interrupt code needs care: use volatile, keep the ISR short, and remember the main code and ISR run asynchronously.